StudySpark – RAG-Based AI Teaching Assistant

Case Study

Project name	StudySpark – RAG-Based AI Teaching Assistant
Project type	AIML | RAG
Domain	Artificial Intelligence (RAG, NLP, Generative AI)
Target users	Students and Self-Learners
Platform	Python + OpenAI Whisper + OpenAI Embeddings / Sentence Transformers
Total hosting cost	₹0 — fully free-tier stack
Academic strength	Improves Learning Efficiency| Context-Aware Answers| Scalable Across Subjects| Enhances Revision
Timeline	Full semester (phased delivery)

1. Executive Summary

StudySpark is an AI-powered teaching assistant built using the Retrieval-Augmented Generation (RAG) architecture. It helps students learn from recorded lectures by providing accurate, context-aware answers based solely on the uploaded course content. Instead of relying on general AI knowledge, StudySpark retrieves relevant information from lecture transcripts before generating responses, significantly improving accuracy and reducing hallucinations.

2. Problem Statement

Students often spend hours searching through long lecture videos to find explanations of specific topics. Traditional chatbots provide generic answers and may not accurately reflect what was taught in a particular course.

The challenge was to build an intelligent assistant capable of:

- Understanding questions in natural language.
- Searching through course lectures efficiently.
- Providing answers grounded in the instructor's content.
- Reducing incorrect or fabricated AI responses.

3. Proposed Solution

StudySpark uses a Retrieval-Augmented Generation (RAG) pipeline to answer questions based on lecture materials.

4. Workflow

5. Technology Stack

The entire **StudySpark** stack is built on free and open-source technologies, with zero hosting cost during development and student deployment phases.

Category	Technology	Purpose	Cost
Programming Language	Python	Core language for implementing the RAG pipeline and backend logic.	Free (open source)
Notebook Environment	Jupyter Notebook	Experimentation, data preprocessing, and model development.	Free
Speech-to-Text	OpenAI Whisper	Converts lecture audio into timestamped text transcripts.	Free (open source)
Audio Processing	FFmpeg	Extracts audio (MP3) from lecture videos.	Free (open source)
Data Processing	Pandas	Cleans, processes, and manages transcript data.	Free
Vector Database	FAISS	Stores embeddings and performs fast semantic similarity search.	Free (open source)
Data Storage	JSON	Free	Free
Code Repository	GitHub	Hosts the project and manages version history.	Free (Public Repositories)
Operating System	Windows	Development and local execution environment.	Windows (Licensed)

6. Implementation Phases

Phase 1 — Video to Audio Conversion

- Extracted audio from lecture videos using Python and FFmpeg.
- Converted videos into MP3 format compatible with the Whisper speech recognition model.

Tools Used: Python, FFmpeg

Phase 2 — Speech-to-Text Transcription

- Processed audio files using OpenAI Whisper.
- Generated accurate transcripts with timestamps for every lecture.
- Stored transcripts in JSON format for further processing.

Tools Used: OpenAI Whisper

Phase 3 — Transcript Chunking

- Divided long transcripts into smaller, meaningful text chunks.
- Preserved timestamps and metadata such as lecture title and chunk ID.
- Optimized chunk size to balance retrieval accuracy and context.

Output: Structured chunks with metadata.

Phase 4 — Embedding Generation

- Converted each text chunk into high-dimensional vector embeddings.
- Captured semantic meaning instead of relying on keyword matching.

Tools Used:

- Sentence Transformers / OpenAI Embeddings

Phase 5 — Vector Database Creation

- Stored all embeddings inside a FAISS vector index.
- Saved processed data using Joblib for faster loading in future sessions.

Tools Used:

- FAISS
- Joblib

Phase 6 — Response Delivery

- Displayed the generated answer to the user.
- Included lecture timestamps (if available) so students could revisit the exact portion of the video.
- Ensured responses remained relevant to the uploaded course content.

7. System Architecture

Data Processing Pipeline

1. Firstly Upload lecture videos.
2. Then Convert videos into audio files.
3. Then Generate transcripts with timestamps using Whisper.
4. Then Split transcripts into meaningful chunks.
5. Then Store metadata.
6. Then Generate embeddings.
7. Then Store embeddings inside FAISS.
8. Then Accept student query.
9. Then Retrieve the most relevant chunks.
10. Then Generates the contextual answer using GPT.

8. Impact

StudySpark transforms passive lecture recordings into an interactive learning experience. Students can ask questions in natural language and receive accurate, context-aware explanations directly from their course content, saving time and improving understanding.

9. Key Learnings

Through this project, I gained hands-on experience with:

- Retrieval-Augmented Generation (RAG)
- Semantic search and vector embeddings
- OpenAI Whisper for speech recognition
- FAISS vector indexing
- Prompt engineering
- Large Language Model integration
- Data preprocessing and chunking strategies

- Building scalable AI pipelines

10. Conclusion

StudySpark successfully demonstrates the implementation of a Retrieval-Augmented Generation (RAG) based AI Teaching Assistant that transforms recorded lecture videos into an interactive learning resource. By integrating speech-to-text transcription, semantic text chunking, vector embeddings, and a Large Language Model, the system provides accurate, context-aware answers based on course-specific content rather than general internet knowledge.

The use of a retrieval mechanism ensures that responses are grounded in the uploaded lecture materials, significantly reducing hallucinations and improving the reliability of AI-generated answers. Additionally, semantic search enables students to quickly locate relevant concepts without manually browsing lengthy video lectures, making the learning process more efficient and engaging.

This project also highlights practical applications of modern AI technologies, including OpenAI Whisper, vector embeddings, FAISS, and Large Language Models, while demonstrating how they can be integrated into a scalable educational platform. The modular design allows StudySpark to support multiple courses and can be extended with features such as PDF integration, multilingual support, voice-based interaction, personalized learning recommendations, quiz generation, and cloud deployment.

In conclusion, StudySpark provides an effective and intelligent solution for enhancing digital education by making lecture content searchable, interactive, and easily accessible. It showcases the potential of Retrieval-Augmented Generation in building reliable AI-powered educational assistants and lays a strong foundation for future advancements in AI-driven learning systems.